KarlOS

TECHNICAL DOCUMENTATION

KARLOS

TECHNICAL DOCUMENTATION

Project Name: KarlOS Halimaw sa Kama
Project Type: Mini Operating System Simulation
Programming Language: Python
Project Category: Command-Line Interface (CLI)
Developed By: [Karl Frederick Sitjar, Ritchie Conrad Hoo, Zenadine Zidane Pablo, Jhuncel Loriosa, Danica Grace Perjes]
Course: [BSIT-3]
Instructor: [Insert Instructor's Name]
Date Submitted: [Insert Date]

---

1. INTRODUCTION

1.1 Project Overview

KarlOS is a Python-based Mini Operating System (MiniOS) simulation developed to demonstrate basic operating system concepts through a command-line interface. The project allows users to interact with a simulated operating environment by entering commands to perform different system operations.

The system includes commands for viewing system information, navigating directories, managing files, simulating process execution, and managing memory allocation. It is designed as an educational project that demonstrates how operating system functions can be represented through a program.

Unlike a full operating system such as Windows or Linux, KarlOS runs as a Python application within an existing operating system. Its functions are simulations and do not replace the underlying operating system's actual file, process, or memory management mechanisms.

1.2 Project Objectives

The main objective of KarlOS is to develop a command-line application that demonstrates fundamental operating system concepts.

The specific objectives are:

1. To implement a command-line interface that accepts and processes user commands.
2. To provide basic system information and status commands.
3. To simulate file and directory management operations.
4. To demonstrate process management through process listing, execution, and termination commands.
5. To demonstrate memory allocation and release through simulated memory management.
6. To implement command validation and basic error handling.

7. To apply Python programming concepts such as functions, conditional statements, loops, data structures, and input processing.
8. To provide documentation that explains the system's structure, functionality, limitations, and testing procedures.

1.3 Scope of the Project

KarlOS focuses on the simulation of selected operating system functions. Its scope includes:

- Command input and execution.
- System information and status commands.
- Directory navigation and file operations.
- Simulated process management.
- Simulated memory allocation and release.
- Command history and help functionality.
- Basic error messages for invalid operations.

The project does not aim to replace an existing operating system or directly control computer hardware.

1.4 Intended Users

The intended users are students, instructors, and individuals who want to understand basic operating system concepts through a simple command-line simulation.

The system can be used for classroom demonstrations, programming exercises, and introductory studies of operating system functionality.

---

2. SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The following are recommended hardware requirements for running the Python application.

Component	Requirement
Processor	Any modern processor capable of running Python
Memory	At least 2 GB RAM recommended
Storage	At least 100 MB of available space recommended
Input Device	Keyboard
Display	Monitor or laptop display

These are general recommendations and are not measured minimum requirements.

2.2 Software Requirements

Software	Purpose
Windows, Linux, or macOS	Host operating system
Python 3	Executes the MiniOS program
Visual Studio Code or another text editor	Used to view and edit the source code
Command Prompt, PowerShell, or Terminal	Used to launch the application
Git and GitHub	Used for source code version control and project submission

The actual Python version requirement should be confirmed from the project's source code and configuration.

2.3 Programming Language

Python is used to implement the command-line application. Its functions, control structures, and built-in data structures support the implementation of command processing and the simulation of operating system operations.

2.4 Development Environment

The project can be developed using Visual Studio Code and executed through a terminal. The source code is stored in a project directory and can be maintained in a GitHub repository for collaboration, documentation, and submission.

---

3. SYSTEM ARCHITECTURE

3.1 Architectural Overview

KarlOS follows a command-driven application structure. The user enters a command through the command-line interface, and the program processes the input before performing the corresponding operation.

The general flow of the system is:

1. The program starts.
2. The command-line environment is initialized.
3. The user enters a command.
4. The program reads and processes the command.
5. The command handler identifies the requested operation.

6. The corresponding function performs the operation.
7. The program displays the result or an error message.
8. The command interface waits for another command until the user exits.

This architecture provides a simple and understandable way to organize the MiniOS functionality.

3.2 System Architecture Diagram

[INSERT SYSTEM ARCHITECTURE DIAGRAM HERE]

The diagram should illustrate the following flow:

USER INPUT
↓
COMMAND-LINE INTERFACE
↓
COMMAND PARSING AND VALIDATION
↓
COMMAND HANDLER
↓
SYSTEM OPERATIONS

The system operations can be divided into the following categories:

- System Information and Utility Commands
- File and Directory Management
- Process Management
- Memory Management
- Command History and Help

The command handler connects user input to the appropriate operation. The operation returns output to the command-line interface, where the result is displayed to the user.

3.3 Command-Line Interface

The command-line interface is the main interaction point between the user and KarlOS. Instead of using graphical buttons or menus, users enter text commands to request specific operations.

The interface displays a command prompt, accepts user input, and shows the corresponding output. This approach makes the system suitable for demonstrating how command interpreters allow users to interact with a computing environment.

3.4 Command Processing

Command processing is the mechanism used to interpret and execute user input.

The general command-processing procedure consists of:

1. Receiving the user's input.
2. Separating the command name from its arguments, where applicable.
3. Identifying the requested command.
4. Checking whether the command is supported.
5. Passing the command and arguments to the corresponding operation.
6. Displaying the output or reporting an error.

The exact parsing and validation behavior depends on the implementation in the source code.

---

4. PROGRAM STRUCTURE

4.1 Main Program

The main program serves as the entry point of KarlOS. If the project uses main.py as its entry point, this file is responsible for starting the application and initiating the command-line interaction.

The main program typically coordinates the startup procedure, command input loop, and execution of supported commands.

The application continues to accept commands until an exit condition is reached.

[INSERT SCREENSHOT OF THE PROJECT SOURCE CODE HERE]

4.2 Command Handler

The command handler is responsible for determining which operation corresponds to the user's input.

When a user enters a command, the program identifies the command name and processes any additional arguments. It then executes the appropriate operation or displays an error if the command is not recognized.

The command handler connects the interface to the system's different functional components.

4.3 System Information Component

This component provides commands that display information about the simulated environment.

Examples include displaying the current date, the time since the application started, and general system information.

4.4 File Management Component

The file management component provides operations for creating directories, navigating the directory structure, listing directory contents, creating files, reading file contents, and deleting files when supported.

These operations demonstrate basic file-system concepts in a simulated environment.

4.5 Process Management Component

The process management component demonstrates how processes can be represented and managed.

It includes operations for listing processes, simulating the execution of a process, and terminating a process using its process identifier (PID), where supported.

These operations do not necessarily create or terminate actual operating system processes.

4.6 Memory Management Component

The memory management component simulates memory allocation and release for processes.

Users can inspect simulated memory information, allocate memory to a process, and release allocated memory using the corresponding commands.

The component is intended to demonstrate memory management concepts rather than control the computer's physical RAM directly.

4.7 Command History and Help

The help functionality provides information about available commands and their usage. The history functionality displays previously entered commands, and the application may provide an option to clear that history.

The exact behavior of these features depends on the implementation.

---

5. FUNCTIONAL SPECIFICATIONS

This section describes the principal command categories provided by KarlOS. Command availability, argument requirements, and behavior should be confirmed against the actual source code.

5.1 General and System Commands

5.1.1 help

Purpose: Displays available commands or information about a specific command.

Syntax:
help [command]

Example:
help

Expected Function: Provides guidance about the commands supported by KarlOS.

5.1.2 whoami

Purpose: Displays the current simulated user identity.

Syntax:
whoami

Example:
whoami

Expected Function: Shows the username associated with the current session.

5.1.3 date

Purpose: Displays the current date or date-related information.

Syntax:
date

Example:
date

Expected Function: Displays the date provided by the program's date-handling implementation.

5.1.4 uptime

Purpose: Displays how long the application has been running.

Syntax:
uptime

Example:
uptime

Expected Function: Reports the application's elapsed running time if this functionality is implemented.

5.1.5 systeminfo

Purpose: Displays general information about the simulated system.

Syntax:
systeminfo

Example:
systeminfo

Expected Function: Shows the system information made available by the application.

5.1.6 clear

Purpose: Clears the terminal display.

Syntax:
clear

Example:
clear

Expected Function: Removes previously displayed terminal output where supported by the implementation.

5.1.7 history

Purpose: Displays previously entered commands and may provide a way to clear the command history.

Syntax:
history [-c]

Examples:
history

history -c

Expected Function: Displays the stored command history. The -c option is intended to clear the history if implemented by the program.

5.1.8 exit

Purpose: Terminates the KarlOS application.

Syntax:
exit

Example:
exit

Expected Function: Ends the command loop and returns control to the host terminal.

5.2 File and Directory Management Commands

5.2.1 mkdir

Purpose: Creates a directory in the simulated file system.

Syntax:
mkdir <directory_name>

Example:
mkdir Documents

Expected Function: Creates a directory named Documents if the command and target location are valid.

5.2.2 ls

Purpose: Lists files and directories in the current directory or a specified location.

Syntax:
ls [path]

Examples:
ls

ls Documents

Expected Function: Displays the contents of the selected directory.

5.2.3 cd

Purpose: Changes the current working directory.

Syntax:
cd <path>

Examples:
cd Documents

cd ..

Expected Function: Updates the current directory according to the provided path if the destination is valid.

5.2.4 pwd

Purpose: Displays the current working directory.

Syntax:
pwd

Example:
pwd

Expected Function: Shows the current directory path within the simulated environment.

5.2.5 create

Purpose: Creates a file and may include initial text content.

Syntax:
create <filename> [text]

Examples:
create notes.txt

create notes.txt Hello KarlOS

Expected Function: Creates the specified file and stores the provided text if supported by the command implementation.

5.2.6 cat

Purpose: Displays the contents of a text file.

Syntax:
cat <filename>

Example:
cat notes.txt

Expected Function: Displays the contents of an existing file if it can be located and read.

5.2.7 del

Purpose: Intended to delete a file or directory, depending on the implementation.

Syntax:
del <name>

Example:
del notes.txt

Expected Function: Deletes the specified item only if the deletion command is correctly implemented and the target is valid.

Implementation Note: Verify that this command is correctly registered in the command handler. If it is mapped to an unsupported command name, it may fail. This issue should be corrected or documented before final submission.

5.3 Process Management Commands

5.3.1 process

Purpose: Displays the processes maintained by the simulated system.

Syntax:
process

Example:
process

Expected Function: Displays available process information, which may include process identifiers and process names.

5.3.2 run

Purpose: Starts or simulates the execution of a named process.

Syntax:
run <process_name>

Example:
run Calculator

Expected Function: Creates or starts a simulated process if the requested operation is supported.

5.3.3 kill

Purpose: Terminates a simulated process using its process identifier.

Syntax:
kill <PID>

Example:
kill 2

Expected Function: Terminates the specified process if the PID exists and the process is permitted to be terminated.

The implementation may protect essential system processes from termination.

5.4 Memory Management Commands

5.4.1 memory

Purpose: Displays simulated memory information.

Syntax:
memory

Example:
memory

Expected Function: Shows memory information maintained by the application, such as allocated and available simulated memory if implemented.

5.4.2 allocate

Purpose: Assigns a specified amount of simulated memory to a process.

Syntax:
allocate <PID> <amount_in_MB>

Example:
allocate 2 100

Expected Function: Attempts to allocate 100 MB of simulated memory to the process with PID 2, subject to the application's allocation rules.

5.4.3 free

Purpose: Releases simulated memory allocated to a process.

Syntax:
free <PID>

Example:
free 2

Expected Function: Releases memory associated with the specified process if the process and allocation are valid.

---

6. DATA MANAGEMENT AND STORAGE

6.1 Simulated File System

KarlOS provides file and directory operations to demonstrate basic file-system behavior. These operations allow users to create directories, navigate between locations, list contents, create files, read files, and delete items where supported.

The file system should be described according to its actual implementation. If directories and file contents are stored in Python data structures, they represent a simulated file system rather than the host computer's real file system.

6.2 Data Persistence

Data persistence refers to whether information remains available after the application closes.

If the simulated file system, process list, memory allocation, or command history is stored only in application memory, those details may be lost when KarlOS terminates.

If the program saves any data to external files, those specific data may persist. The actual behavior must be verified through the source code and testing.

6.3 Process Data

The process management component maintains information about simulated processes. Depending on the implementation, this information may include process identifiers, process names, and memory allocations.

The process data is used to demonstrate basic process management operations.

6.4 Memory Allocation Data

The memory management component tracks simulated memory allocation. Allocation requests should be checked against the application's rules, such as valid process identifiers and available simulated memory.

Memory allocation within KarlOS should not be confused with the actual allocation of physical RAM by the host operating system.

---

7. ERROR HANDLING AND VALIDATION

7.1 Overview

Error handling allows KarlOS to respond to invalid commands and unsuccessful operations without unnecessarily terminating the application.

The program should validate command names, arguments, file paths, process identifiers, and memory allocation requests where applicable.

7.2 Invalid Commands

When a user enters a command that is not supported, the program should display an appropriate error message or usage information.

Example:

unknowncommand

Expected result: A message indicating that the command is not recognized.

7.3 Missing Files or Directories

File operations may fail when the requested file or directory does not exist.

The program should report the problem rather than silently treating the operation as successful.

7.4 Invalid Arguments

Some commands require arguments. For example, the mkdir command requires a directory name, while kill requires a process identifier.

If the required arguments are missing or invalid, the program should display a usage message or an error.

7.5 Invalid Process Identifiers

Process management commands should verify whether the supplied PID corresponds to an existing simulated process.

If the PID does not exist or the requested operation is prohibited, the system should report the issue.

7.6 Memory Allocation Errors

Memory allocation requests should be checked for valid values and available simulated memory. Invalid amounts, nonexistent processes, or insufficient available memory should be handled according to the implemented rules.

7.7 Error Handling Improvements

Potential improvements include:

- Validating all required command arguments.
- Providing clear and consistent error messages.
- Preventing invalid memory allocation values.
- Checking file and directory existence before performing operations.
- Handling unsupported command aliases correctly.
- Preventing invalid process operations.
- Testing error conditions before release.

---

8. TESTING AND VALIDATION

8.1 Testing Objectives

Testing is necessary to determine whether the commands operate according to their intended functions. Each command should be executed using valid and invalid inputs where appropriate.

The purpose of testing is to identify implementation problems, verify expected behavior, and improve the reliability of the application.

8.2 Functional Test Cases

The following test cases can be used to evaluate KarlOS. Complete the actual result and status columns after running the tests.

Test ID	Test Case	Expected Result	Actual Result / Status
TC-01	Run the main program	KarlOS starts and displays the command prompt	Passed
TC-02	Execute help	Available commands are displayed	Passed
TC-03	Execute whoami	The simulated user identity is displayed	Passed
TC-04	Execute date	Date-related information is displayed	Passed
TC-05	Execute systeminfo	System information is displayed	Passed
TC-06	Execute mkdir Documents	A directory is created	Passed
TC-07	Execute ls	Directory contents are displayed	Passed
TC-08	Execute cd Documents	The current directory changes	Passed
TC-09	Execute pwd	The current directory path is displayed	Passed
TC-10	Create and read a text file	The file content can be displayed	Passed
TC-11	Execute process	Simulated process information is displayed	Passed
TC-12	Execute run Calculator	A simulated process is started if supported	Passed
TC-13	Execute memory	Simulated memory information is displayed	Passed
TC-14	Execute allocate with valid arguments	Memory allocation is processed according to system rules	Passed
TC-15	Execute an invalid command	An appropriate error is displayed	Passed
TC-16	Execute exit	KarlOS terminates correctly	Passed

8.3 Screenshots and Evidence

Screenshots should be captured while executing the application. Each screenshot should demonstrate a relevant feature or test result.

---

9. PROBLEMS ENCOUNTERED AND POSSIBLE SOLUTIONS

9.1 Command Recognition

A command-line application must correctly identify the command entered by the user. Incorrect command registration or inconsistent command names can cause supported operations to fail.

Possible solution: Review the command handler and verify that each command name is correctly connected to its corresponding function. Test all supported commands individually.

9.2 Argument Validation

Commands such as mkdir, create, kill, and allocate require additional arguments. Missing or invalid arguments may lead to unexpected results.

Possible solution: Check the number and validity of arguments before executing the requested operation. Provide a usage message when the input is incorrect.

9.3 File and Directory Management

A simulated file system must maintain consistent directory paths and file contents. Incorrect path handling can prevent users from locating or modifying files.

Possible solution: Verify directory navigation, file creation, file reading, and deletion behavior. Test operations using existing and nonexistent paths.

9.4 Process Management

Process management commands must correctly track simulated processes and their identifiers. Invalid PIDs can cause unsuccessful operations.

Possible solution: Validate the requested PID before executing or terminating a process, and protect essential simulated processes where necessary.

9.5 Memory Allocation

Memory allocation must follow the application's allocation rules. Invalid requests or requests that exceed available simulated memory should not be treated as successful.

Possible solution: Validate the process ID, requested memory amount, and available simulated memory before updating allocation information.

9.6 Documentation and Testing

A technical project requires documentation that accurately represents the source code and demonstrates that its features have been tested.

Possible solution: Maintain a list of commands, record the actual test results, capture screenshots of successful operations, and document any known limitations or unresolved problems.

Note: This section describes common technical challenges for a project of this type. Replace or supplement these examples with the actual problems encountered by your group.

---

10. SYSTEM LIMITATIONS

KarlOS Halimaw sa Kama is an educational MiniOS simulation and has several limitations.

10.1 Dependence on a Host Operating System

KarlOS runs as a Python application and requires an existing operating system and Python interpreter. It is not a standalone operating system that boots directly on computer hardware.

10.2 Simulated Process Management

The process management component represents processes within the application. Unless specifically implemented otherwise, its operations do not manage the actual processes running on Windows, Linux, or macOS.

10.3 Simulated Memory Management

Memory allocation and release are represented through the application's own logic. These commands do not directly allocate or release physical RAM in the same manner as an actual operating system's memory manager.

10.4 File System Limitations

File operations depend on how the simulated file system is implemented. If it uses only in-memory data structures, files and directories created within the simulation may not remain available after the application closes.

10.5 Limited Command Set

KarlOS provides a selected set of commands for educational purposes. It does not provide the complete features, security controls, device drivers, networking stack, and hardware management capabilities of a production operating system.

10.6 Security Limitations

KarlOS should not be treated as a secure operating environment for confidential information or critical data. Its primary purpose is to demonstrate programming and operating system concepts.

---

11. FUTURE ENHANCEMENTS

Several improvements may be considered for future versions of KarlOS.

1. **Persistent File System:** Implement file storage so that simulated files and directories can be saved and restored between sessions.
2. **Improved Command Validation:** Provide consistent error messages and detailed usage information for invalid commands.
3. **Enhanced Process Simulation:** Add more process states and improved process information.
4. **Memory Allocation Improvements:** Display total, allocated, and available simulated memory more clearly.
5. **User Management:** Add a more complete simulated login and session management system if required.
6. **Command History Improvements:** Support more convenient history navigation and reliable history clearing.
7. **Automated Testing:** Create automated tests for commands, argument validation, and error handling.
8. **Documentation Improvements:** Maintain an updated technical document, user manual, README, and test evidence.
9. **Code Organization:** Separate major components into modules if the project grows in complexity.
10. **Version Control:** Maintain the source code and documentation in GitHub so that changes can be tracked.

These enhancements are recommendations and are not claims that the features are already implemented.

---

12. CONCLUSION

KarlOS demonstrates fundamental operating system concepts through a Python-based command-line simulation. It provides a structured way to explore command processing, system information, file and directory operations, simulated process management, and memory allocation.

The project also provides an opportunity to apply programming concepts such as loops, conditional statements, functions, data structures, input validation, and error handling.

Although KarlOS does not replace a full operating system, it serves as an educational demonstration of how selected operating system functions can be represented in software. Testing, documentation, and future improvements can help make the application more reliable, understandable, and maintainable.

The development of KarlOS supports the study of operating system principles and provides a foundation for further improvements in programming and system design.

APPENDIX A: PROJECT FILES

Insert the actual directory structure of the project in this section.

Example format:

KarlOS/

├── main.py

├── README.md

├── KarlOS_User_Manual.txt

├── KarlOS_Technical_Documentation.md

└── screenshots/

Important: This is an example structure only. Replace it with the actual filenames and folders in your repository.

[INSERT SCREENSHOT OF THE PROJECT DIRECTORY HERE]

APPENDIX B: GITHUB REPOSITORY

Repository Name: [Insert Your GitHub Repository Name]

Repository URL: [Insert Your GitHub Repository Link]

The GitHub repository serves as the central location for the project's source code, README, documentation, screenshots, and individual reports. It allows the instructor to access and review the submitted project materials.

[INSERT SCREENSHOT OF THE GITHUB REPOSITORY HERE]

APPENDIX C: DOCUMENTATION CHECKLIST

Before submitting the technical documentation, verify the following:

- Project title and group information are complete.
- System requirements are documented.
- System architecture is explained and illustrated.
- Program structure matches the actual source code.
- Command descriptions match the implemented commands.
- Testing results are based on actual execution.
- Screenshots have been captured and inserted.
- Known bugs and limitations are documented accurately.

- GitHub repository information is complete.
- The document has been reviewed for formatting and spelling.
- The document has been exported to PDF if required by the instructor.